DIBRIS

DEPARTMENT OF INFORMATICS,
BIOENGINEERING, ROBOTICS AND SYSTEM ENGINEERING

ARTIFICIAL INTELLIGENCE FOR ROBOTICS II 2025/26

Assignment D4-V2

Search and Rescue - Unknown Victim Location

Author:

Edoardo Ruggeri

Professor:

Fulvio Mastrogiovanni

Omar Kashmar

1 Introduction

Scenario

The objective of this assignment is to model a search-and-rescue planning problem in which a single mobile robot operates inside a damaged building. The topology of the environment is known, but the victim location is initially unknown to the robot. The robot must therefore explore the building, inspect rooms, detect the victim, and execute a rescue strategy before time-dependent health degradation makes the mission fail.

Search-and-Rescue Domain

The operational capabilities and state-transition logic of the rescue robot are formalized through PDDL and PDDL+. The model is built around the following components:

- **Type and object structure:** the domain separates the mobile robot, rooms, the safe base, and the victim/patient-related state. The robot used in the instances is named `rescuebot`.
- **State predicates:** Boolean predicates encode topological position (`robot-at`), room connectivity (`connected`), inspection status (`inspected`), actual victim location (`victim-at`), acquired information (`victim-detected`), and rescue completion (`rescued`).
- **Causal pipeline:** the robot cannot rescue the victim immediately. Valid plans must follow the dependency chain: `move` -> `inspect` -> `detect` -> `rescue`.
  In the extended models, this becomes `detect victim -> assess health -> choose branch -> transport/stabilize`.
- **Temporal pressure:** in the PDDL+ models, victim health is represented as a numeric fluent and decreases continuously while the mission is active. Delays caused by exploration, waiting, or transport can therefore make a symbolic path infeasible.

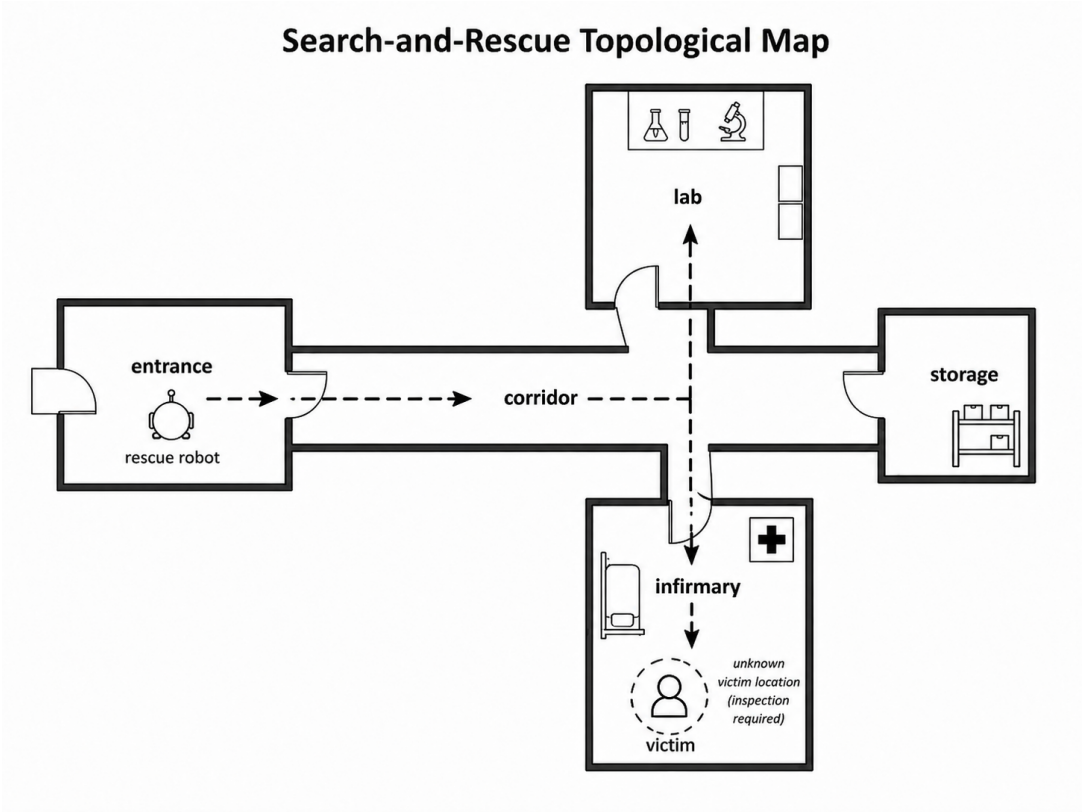

Figure 1: Schematic representation of the search-and-rescue topological environment.

Planner

The PDDL+ models were validated with **ENHSP local Docker**. This local setup was used to avoid remote Planning Domains package-discovery issues and to obtain repeatable validation through the VS Code PDDL workflow. Classical PDDL instances were checked through planner execution and saved output files.

2 Core Assignment Model

The core deliverable is stored in `codes/D4-V2_search_and_rescue/`. It contains the official Q1 and Q2 material: a Basic PDDL model for exploration and a PDDL+ extension for time-dependent health degradation. The core version interprets *rescue* as an on-site action enabled after victim detection.

2.1 Q1 - Basic PDDL

Q1 approximates unknown victim location using deterministic symbolic inspection. Classical PDDL requires the initial state to be fully specified, so the true victim location is still encoded in the problem file. The model prevents direct rescue by separating world reality from robot knowledge:

Layer	Predicate	Meaning
World state	`(victim-at ?loc)`	The victim is actually located in a room in the fully specified model.
Robot knowledge	`(victim-detected ?loc)`	The robot has acquired the information through inspection.
Goal logic	`(rescued)`	Rescue can be achieved only after detection.

The main Q1 actions are `move`, `inspect-empty-room`, `inspect-victim-room`, and `rescue`. The exploration problem forces the robot to inspect rooms before rescue becomes applicable. This is not true partial observability, but it is a controlled approximation consistent with the Basic PDDL requirement.

Listing 1: Core symbolic separation between world truth and acquired information.

```
;; Real fact required by the fully specified classical PDDL state.
(victim-at infirmary)

;; Information acquired by the robot after inspecting the victim room.
(victim-detected infirmary)

;; The rescue action depends on victim-detected, not directly on victim-at.
```

2.2 Q2 - PDDL+ Extension

Q2 extends the same domain with a time-dependent model of victim health. The important change is that the world evolves even when the planner is not applying a discrete rescue action. The robot must now solve not only a reachability problem, but also a feasibility problem under a health deadline.

PDDL+ construct	Role in the rescue scenario
`victim-health`	Numeric fluent representing remaining patient health.
`health-degradation`	Process that decreases health continuously while the mission is active.
`victim-dies`	Event triggered automatically when health reaches zero.
Fast rescue instance	Demonstrates that rescue succeeds if detection and rescue happen quickly enough.
Delayed rescue instance	Demonstrates that additional waiting can invalidate an otherwise reachable rescue sequence.

Listing 2: PDDL+ process and event used to model time pressure.

```
(:process health-degradation
 :precondition (and (mission-active) (victim-alive) (> (victim-health) 0))
 :effect (decrease (victim-health) (* #t 1)))

(:event victim-dies
 :precondition (and (mission-active) (victim-alive) (<= (victim-health) 0))
 :effect (and (victim-dead) (not (victim-alive)) (not (mission-active))))
```

3 Alternative Interpretations of Rescue

The repository intentionally contains three related versions of the scenario because the word *rescue* is not fully specified in the assignment statement. Three reasonable interpretations were considered:

Version	Interpretation of rescue	Role in the project
Core assignment	On-site rescue action after victim detection.	Required Q1/Q2 deliverable.
Transport variant	Evacuation to base: load patient, move while carrying, unload at the safe location.	Extra model exploring a stronger interpretation.
Definitive model	Health-aware rescue: direct transport, stabilization first, or failure if too late.	Final extended model and recommended version to inspect first.

The corresponding folders are `codes/D4-V2_search_and_rescue/`, `codes/D4-V2_search_and_rescue_variant/` and `codes/D4-V2_search_and_rescue_definitive/`.

3.1 Transport-to-Base Variant

The transport variant strengthens the meaning of rescue. The goal is not only that the victim has been helped on site, but that the patient has been physically evacuated to the base. This adds actions such as `start-load-patient`, `start-move-with-patient`, and `start-unload-patient-at-base`. This version is not a replacement for the core deliverable; it documents how a different reading of the word *rescue* changes the causal structure of the plan.

3.2 Definitive Model

The definitive version is the final extended model and the recommended folder to inspect first. It combines exploration, victim detection, patient assessment, health degradation, transport, stabilization, mission deadline logic, and a too-late unsolvable case. Its high-level pipeline is:

`search -> inspect -> detect victim -> assess patient -> choose branch -> complete mission.`

The branch is selected declaratively by PDDL/PDDL+ preconditions, not by external procedural code.

4 PDDL+ Domain Analysis

The PDDL+ model introduces continuous change, threshold events, and numeric feasibility checks. The central numeric quantity is `victim-health`. Since the degradation process uses `#t`, health loss depends on elapsed time rather than on the number of symbolic actions alone.

4.1 Health-Based Branch Selection

In the definitive model, the robot assesses the victim and then follows the branch made applicable by the current health level and mission timing.

Condition	Planner branch	Expected behaviour
Health high enough	Direct transport	The robot loads and transports the patient without stabilization.
Health low but recoverable	Stabilize then transport	The robot stabilizes the patient before evacuation.
Health zero, too low, or mission too late	No valid rescue plan	The failure/deadline condition prevents a valid solution.

Stabilization is an additional modelling layer beyond the minimum assignment. It represents the idea that a low-health patient may not be safely transportable without prior intervention. This turns the model from a simple evacuation problem into a health-aware planning problem.

4.2 Mission Deadline

During testing, the planner could exploit artificial waiting. Since health decreases continuously, arbitrary waiting could push the model below a threshold and force stabilization even when direct transport should be selected immediately. The definitive version therefore includes `mission-time`, `mission-deadline`, a `mission-clock` process, and a `mission-timeout` event. This makes branch selection meaningful and prevents waiting from becoming a modelling artefact.

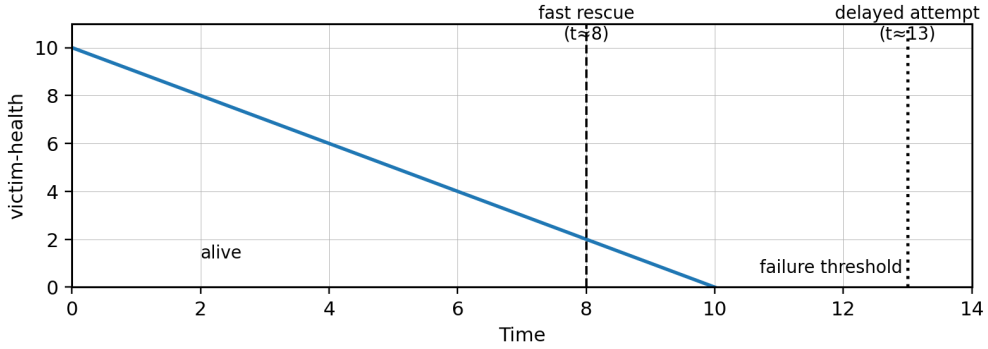

Figure 2: Schematic health trend: the fast rescue succeeds before the health threshold is reached, while the delayed attempt exceeds the available health horizon.

5 Validation and Planner Results

The final repository stores planner outputs inside the corresponding model folders. Validation was performed by checking both syntactic planner output and semantic consistency with the goal conditions. This distinction matters because some PDDL+ planner outputs may be syntactically returned while still being semantically suspicious, for example when an empty plan is returned for a non-empty goal whose predicates are false in the initial state.

Problem file	Model, saved output, and validated result
problem_known_location.pddl	**Core Q1**. Saved in plan_known_location.txt. Valid plan; rescue occurs only after detection.
problem_exploration.pddl	**Core Q1**. Saved in plan_exploration.txt. Valid plan; room inspection is required before rescue.
problem_fast_rescue.pddl	**Core Q2**. Saved in plan_fast_rescue.txt. Successful output; health remains above zero.
problem_delayed_rescue.pddl	**Core Q2**. Saved in plan_delayed_rescue.txt. Documented delayed failure; forced waiting makes rescue semantically invalid.
problem_variant_fast.pddl	**Variant**. Saved in plan_variant_fast.txt. Fast evacuation validated; the patient is loaded, transported, and unloaded at base.
problem_variant_delayed.pddl	**Variant**. Saved in plan_variant_delayed.txt. Delayed evacuation documented; transport-to-base becomes impossible or invalid.
problem_definitive_direct_transport.pddl	**Definitive**. Saved in plan_definitive_direct_transport.txt. Direct branch solved; start-stabilize-patient does not appear.
problem_definitive_stabilize_then_transport.pddl	**Definitive**. Saved in plan_definitive_stabilize_then_transport.txt. Stabilization branch solved; start-stabilize-patient appears before transport.
problem_definitive_too_late.pddl	**Definitive**. Saved in plan_definitive_too_late.txt. Correctly unsolvable; rescue is impossible before failure/deadline conditions.

The most important result is that the model distinguishes three outcomes: rescue before health failure, rescue after stabilization, and correct unsolvability when the mission starts too late.

6 Final Discussion

The project satisfies the official assignment requirements and also records the modelling evolution caused by the ambiguity of the term *rescue*. The core model provides the required Basic PDDL approximation and PDDL+ health-degradation extension. The transport variant shows how the model changes if rescue means evacuation. The definitive model combines the previous ideas and adds health-based branch selection.

Partial Observability

The natural-language scenario is partially observable, because the robot does not initially know where the victim is. Classical PDDL does not represent hidden state, belief distributions, or probabilistic sensing. The project therefore approximates sensing with deterministic inspection actions and separates victim-at from victim-detected. This is acceptable for the required Basic PDDL model, but it remains a limitation: a full treatment would require belief-space planning or a POMDP-like formalization.

Sensing and Time

PDDL+ captures the main interaction of the scenario: sensing consumes time, and time affects survival. Movement, inspection, waiting, transport, and stabilization are no longer neutral with respect to mission success, because victim health decreases continuously while the mission is active. The result is that a plan can be symbolically valid but temporally infeasible.

Limitations and Extra Work

The model remains abstract: there is no geometric motion planning, no noisy perception, no probabilistic belief update, no execution uncertainty, and no realistic medical dynamics. These limitations are deliberate consequences of using a tractable PDDL/PDDL+ abstraction. The additional transport and definitive versions are included because they clarify modelling choices rather than duplicating the same solution. The final definitive version is therefore the richest model, while the core folder remains the official Q1/Q2 assignment deliverable.